



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai - 400058

Model Solutions / Synoptic

Course: Deep Learning

Course Code: IT13

Mid Semester Examination October 2025

Max Marks: 30

Q. 1a Let $f(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$. Derive the gradient descent update rule and prove convergence requires $\eta < \frac{2}{\lambda_{\max}}$, where $\lambda_{\max}$ is the largest eigenvalue of $\mathbf{X}^T\mathbf{X}$.

Solution:

Let $\mathbf{X}$ is $m \times n$, $\mathbf{w} \in \mathbb{R}^n$, $\mathbf{y} \in \mathbb{R}^m$. We want the gradient $\nabla_{\mathbf{w}}f$, the gradient descent update, and the step-size condition for convergence.

Step 1 — write the loss explicitly.

$$f(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = \frac{1}{2}(\mathbf{X}\mathbf{w} - \mathbf{y})^T(\mathbf{X}\mathbf{w} - \mathbf{y}).$$

This is a scalar function.

Step 2 — compute gradient Differentiate w.r.t. $\mathbf{w}$:

$$\nabla_{\mathbf{w}}f = \frac{1}{2} \cdot 2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}).$$

(Reason: treat $\mathbf{X}\mathbf{w}$ as a linear function of $\mathbf{w}$; derivative of $\mathbf{w}^T A \mathbf{w}$ is $(A + A^T)\mathbf{w}$, here $A = \frac{1}{2}\mathbf{X}^T\mathbf{X}$ symmetric.)

Step 3 — gradient descent update. Gradient descent with step size η iterates:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla f(\mathbf{w}_t) = \mathbf{w}_t - \eta \mathbf{X}^T(\mathbf{X}\mathbf{w}_t - \mathbf{y}).$$

Step 4 — write as linear iteration around optimum. Define $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ and $\mathbf{b} = \mathbf{X}^T\mathbf{y}$. The normal equations give $\mathbf{A}\mathbf{w}^* = \mathbf{b}$ (least squares solution). Subtract $\mathbf{w}^*$ from both sides of the update:

$$\mathbf{w}_{t+1} - \mathbf{w}^* = \mathbf{w}_t - \mathbf{w}^* - \eta \mathbf{A}(\mathbf{w}_t - \mathbf{w}^*).$$

So the error vector $\mathbf{e}_t := \mathbf{w}_t - \mathbf{w}^*$ evolves as:

$$\mathbf{e}_{t+1} = (\mathbf{I} - \eta \mathbf{A})\mathbf{e}_t.$$

Step 5 — analyse convergence by diagonalizing $\mathbf{A}$. $\mathbf{A}$ is symmetric positive semidefinite (because $\mathbf{A} = \mathbf{X}^T\mathbf{X}$). Therefore it has an orthogonal eigendecomposition:

$$\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T,$$

with $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$, $\lambda_i \geq 0$, and $\mathbf{Q}\mathbf{Q}^T = \mathbf{I}$.

Change coordinates to eigenbasis: $\mathbf{u}_t = \mathbf{Q}^T \mathbf{e}_t$. Then

$$\mathbf{u}_{t+1} = \mathbf{Q}^T \mathbf{e}_{t+1} = \mathbf{Q}^T (\mathbf{I} - \eta \mathbf{A}) \mathbf{Q} \mathbf{u}_t = (\mathbf{I} - \eta \mathbf{\Lambda}) \mathbf{u}_t.$$

This means each coordinate $u_{t,i}$ evolves independently:

$$u_{t+1,i} = (1 - \eta \lambda_i) u_{t,i}.$$

Step 6 — convergence condition per coordinate. For a scalar iteration $u_{t+1} = \alpha u_t$, convergence to 0 occurs iff $|\alpha| < 1$. So require $|1 - \eta \lambda_i| < 1$ for every i , solve:

$$-1 < 1 - \eta \lambda_i < 1 \quad \Rightarrow \quad 0 < \eta < \frac{2}{\lambda_i}.$$

Since this must hold for all eigenvalues, we require

$$0 < \eta < \frac{2}{\lambda_{\max}}.$$

Step 7 — small numeric example to build intuition. Let $\mathbf{A} = \text{diag}(1, 4)$. Then $\lambda_{\max} = 4$. If $\eta = 0.6$, then $1 - \eta \lambda_1 = 0.4$ and $1 - \eta \lambda_2 = -1.4$. The second factor has magnitude > 1 so the corresponding component will oscillate and grow. If we choose $\eta = 0.4$ then both factors are within $(-1, 1)$ and converge.

Marking guideline (5 marks):

- Gradient derivation (2 marks) — show $\nabla f = X^T(Xw - y)$.
- Iteration and eigen-decomposition steps (2 marks).
- Final inequality and explanation (1 mark).

Q.1b Derive the bias-variance decomposition for the expected squared error of a predictor. Using this framework, explain why overfitting occurs when model capacity is too high and how it affects generalization.

Solution:

Let target function $f^*(x)$ and noise ε with $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$. Observed $y = f^*(x) + \varepsilon$. Predictor (trained on dataset) is $\hat{f}(x)$ which is a random variable because training data is random. We compute expected squared error at a fixed x :

$$\mathbb{E}_{\text{data}, \varepsilon} [(y - \hat{f}(x))^2].$$

Step 1 — expand and separate terms.

$$\begin{aligned} \mathbb{E}[(y - \hat{f}(x))^2] &= \mathbb{E}[(f^*(x) + \varepsilon - \hat{f}(x))^2] \\ &= \mathbb{E}[(f^*(x) - \hat{f}(x))^2] + \mathbb{E}[\varepsilon^2] + 2\mathbb{E}[\varepsilon(f^*(x) - \hat{f}(x))]. \end{aligned}$$

Because ε is independent and zero mean, the mixed term is zero: $\mathbb{E}[\varepsilon(f^*(x) - \hat{f}(x))] = 0$. So:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \mathbb{E}[(\hat{f}(x) - f^*(x))^2] + \sigma^2.$$

Step 2 — decompose the first term into bias and variance. Use identity for any random variable Z and constant a :

$$\mathbb{E}[(Z - a)^2] = (\mathbb{E}[Z] - a)^2 + \mathbb{E}[(Z - \mathbb{E}[Z])^2].$$

Set $Z = \hat{f}(x), a = f^*(x)$:

$$\mathbb{E}[(\hat{f}(x) - f^*(x))^2] = (\mathbb{E}[\hat{f}(x)] - f^*(x))^2 + \text{Var}(\hat{f}(x)).$$

So finally:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\text{Bias}[\hat{f}(x)])^2}_{(\mathbb{E}[\hat{f}] - f^*)^2} + \underbrace{\text{Var}(\hat{f}(x))}_{\text{due to training data}} + \sigma^2.$$

Step 3 — connect to overfitting / underfitting. - If model capacity is small (too simple), bias term dominates: systematic error large (underfitting).

- If model capacity is very large, bias decreases (model can fit training function) but variance increases (model fits noise differently across training sets) — this is *overfitting*. The test error increases because variance term dominates. The irreducible noise σ^2 cannot be reduced.

Step 4 — remedies To control variance: regularization, more training data, ensembling, early stopping, dropout. To control bias: richer model, more features, deeper networks.

Marking guideline (5 marks):

- Clear expansion and decomposition (3 marks).
- Explanation linking capacity $\rightarrow$ bias/variance $\rightarrow$ generalization (2 marks).

Q.2a Derive the computational complexity (number of multiplications) of a standard 2D convolution vs. a depthwise separable convolution for input size $H \times W \times C$ with K filters of size $F \times F$. Why are depthwise separable convolutions critical in modern architectures? Provide a theoretical justification in terms of parameter efficiency.

Solution:

Step 1 — standard convolutional layer detailed counting. - Input: height H , width W , channels C .

- We apply K filters; each filter is of size $F \times F \times C$.
- For one output spatial position and one filter: $F^2 \times C$ multiplications.
- Number of output spatial positions approx $H \times W$ (assuming output size same for simplicity).
- So total multiplications:

$$M_{\text{std}} = HW \cdot K \cdot (CF^2).$$

- Parameters: $P_{\text{std}} = K \cdot C \cdot F^2$.

Step 2 — depthwise separable convolution (two-stage). Depthwise separable convolution = depthwise convolution + pointwise convolution (1×1 convolution).

Depthwise stage: - For each of C channels we have an $F \times F$ filter (no cross-channel mixing).

- Multiplications: $HW \cdot C \cdot F^2$.

- Parameters: $C \cdot F^2$.

Pointwise stage: - Use K filters of size $1 \times 1 \times C$ to mix channels.

- Multiplications: $HW \cdot K \cdot C$.

- Parameters: $K \cdot C$.

Total multiplications:

$$M_{\text{sep}} = HW \cdot (CF^2 + KC) = HW \cdot C(F^2 + K).$$

Total parameters:

$$P_{\text{sep}} = CF^2 + KC.$$

Step 3 — ratio and numerical intuition. Ratio (reduction):

$$\frac{M_{\text{sep}}}{M_{\text{std}}} = \frac{CF^2 + KC}{KCF^2} = \frac{1}{K} + \frac{1}{F^2}.$$

Example: $F = 3$, $K = 128$: ratio $\approx 1/128 + 1/9 \approx 0.112$, i.e., roughly $9 \times$ speedup.

Step 4 — theoretical and practical importance. - Depthwise separable convolutions separate spatial filtering and channel mixing: this reduces the number of free parameters and computation while preserving the representational essence when stacked.

- In mobile/embedded settings, this is critical (MobileNet family).

- Lower parameters $\rightarrow$ lower overfitting risk for given data, faster inference.

Marking guideline (5 marks):

- Correct derivation of M_{std} (1.5 marks) and M_{sep} (1.5 marks).
- Ratio and numerical example (1 mark).
- Explanation of practical importance (1 mark).

Q.2b Prove that a linear undercomplete autoencoder trained with MSE learns a subspace equivalent to PCA. Further, explain why an overcomplete autoencoder without regularization tends to learn the identity function.

Solution:

Step 1 — setup linear autoencoder. - Data matrix $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_N]^T$ (rows are data-points). Assume data is zero-centered (subtract mean).

- Encoder weight matrix: $\mathbf{W}_e \in \mathbb{R}^{k \times d}$.

- Decoder weight matrix: $\mathbf{W}_d \in \mathbb{R}^{d \times k}$.

- Reconstruction: $\hat{\mathbf{x}} = \mathbf{W}_d \mathbf{W}_e \mathbf{x}$.
- Loss (MSE over dataset):

$$L(\mathbf{W}_e, \mathbf{W}_d) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \mathbf{W}_d \mathbf{W}_e \mathbf{x}_i\|^2 = \|\mathbf{X} - \mathbf{X} \mathbf{M}^T\|_F^2,$$

where $\mathbf{M} = \mathbf{W}_d \mathbf{W}_e$ is $d \times d$ but of rank at most k .

Step 2 — connect to low-rank approximation (Eckart-Young). Minimizing $\|\mathbf{X} - \mathbf{M}\mathbf{X}\|_F^2$ over matrices $\mathbf{M}$ of rank $\leq k$ is equivalent to approximating the row-space of $\mathbf{X}$ by a rank- k linear operator. By the Eckart-Young theorem (SVD-based optimal low-rank approximation), the best rank- k linear approximation is projection onto the top- k principal components. Therefore $\mathbf{M}$ that minimizes the loss is the projector onto the top- k PCA subspace.

Step 3 — constructive viewpoint. - Let SVD: $\mathbf{X} = \mathbf{U}\Sigma\mathbf{V}^T$. Top- k principal directions are columns of $\mathbf{V}_k$. - Choose $\mathbf{W}_e = \mathbf{V}_k^T$ (encoder projects onto top- k PCs) and $\mathbf{W}_d = \mathbf{V}_k$ (decoder reconstructs from projection). Then $\mathbf{W}_d \mathbf{W}_e = \mathbf{V}_k \mathbf{V}_k^T$ (orthogonal projector). This is optimal.

Step 4 — why overcomplete autoencoder learns identity if unconstrained. If $k \geq d$ then $\mathbf{W}_e$ and $\mathbf{W}_d$ can be chosen such that $\mathbf{W}_d \mathbf{W}_e = \mathbf{I}_d$ (identity operator). For example, set $\mathbf{W}_e = [\mathbf{I}_d, \mathbf{0}]^T$ and $\mathbf{W}_d = [\mathbf{I}_d, \mathbf{0}]$ appropriately. Then $\hat{\mathbf{x}} = \mathbf{x}$ and loss zero. Hence model learns identity mapping — not useful for compression.

Step 5 — how to avoid identity in overcomplete case. Use regularization:

- **Sparsity penalty:** add $\lambda \sum_j \text{KL}(\rho \|\hat{\rho}_j)$ forcing average activation to be small.
- **Denoising AE:** train to reconstruct clean $\mathbf{x}$ from noisy input, forcing robust features.
- **Contractive AE:** penalty on Jacobian norm $\|\frac{\partial h}{\partial x}\|_F^2$.

These constraints prevent trivial identity mapping and encourage meaningful features.

Marking guideline (5 marks):

- Clear mapping from AE loss to low-rank approximation and SVD/PCA argument (3 marks).
- Explanation of identity for overcomplete case and remedies (2 marks).

Q.3a Derive the recursive gradient expression $\frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t-k}}$ for a RNN. Then, explain how the forget gate in an LSTM modifies this recursion and prevents vanishing gradients under suitable initialization.

Solution:

Step 1 — vanilla RNN forward equations. Let

$$\mathbf{a}_t = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}, \quad \mathbf{h}_t = \phi(\mathbf{a}_t),$$

where ϕ is elementwise (tanh or sigmoid). Suppose loss $\mathcal{L}$ is scalar and depends on outputs which depend on $\mathbf{h}_t$; general BPTT applies.

Step 2 — single-step chain rule. We want $\frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t-k}}$. By chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t-k}} = \sum_{s=t-k}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_s} \frac{\partial \mathbf{h}_s}{\partial \mathbf{h}_{t-k}}.$$

(When considering contribution to loss at times $\geq t-k$. For simplicity, focus on one particular target time t so formula becomes product of Jacobians back to $t-k$; this captures the essential vanishing/exploding behavior.)

Step 3 — Jacobian between consecutive hidden states. Compute:

$$\frac{\partial \mathbf{h}_s}{\partial \mathbf{h}_{s-1}} = \frac{\partial \phi(\mathbf{a}_s)}{\partial \mathbf{a}_s} \cdot \frac{\partial \mathbf{a}_s}{\partial \mathbf{h}_{s-1}} = \text{diag}(\phi'(\mathbf{a}_s)) \cdot \mathbf{W}_h.$$

Denote $\mathbf{J}_s := \text{diag}(\phi'(\mathbf{a}_s)) \mathbf{W}_h$.

Step 4 — product for multi-step derivative. Therefore

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-k}} = \mathbf{J}_t \mathbf{J}_{t-1} \cdots \mathbf{J}_{t-k+1} = \prod_{s=t-k+1}^t \mathbf{J}_s.$$

Using chain rule:

$$\boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t-k}} = \left(\prod_{s=t-k+1}^t \mathbf{J}_s \right)^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t}}.$$

(Transpose placement depends on row/column gradient conventions; above expression shows the product structure that matters.)

Step 5 — norm bound leads to vanishing/exploding. Take norms:

$$\left\| \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t-k}} \right\| \leq \left\| \prod_{s=t-k+1}^t \mathbf{J}_s \right\| \cdot \left\| \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \right\| \leq \prod_{s=t-k+1}^t \|\mathbf{J}_s\| \cdot \left\| \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \right\|.$$

If each $\|\mathbf{J}_s\| \leq \rho < 1$ then the product decays like ρ^k (exponential decay) $\Rightarrow$ vanishing gradients. If $\|\mathbf{J}_s\| > 1$ then gradients explode.

Note: $\|\mathbf{J}_s\|$ depends on $\|\mathbf{W}_h\|$ and the maximum of $|\phi'|$. For tanh, $|\phi'| \leq 1$. So spectral radius $\rho(\mathbf{W}_h)$ is critical.

Step 6 — LSTM cell and forget gate (how it helps). LSTM has cell state c_t and gates:

$$\begin{aligned} f_t &= \sigma(\cdot) \quad (\text{forget gate}) \\ i_t &= \sigma(\cdot) \quad (\text{input gate}) \\ \tilde{c}_t &= \tanh(\cdot) \quad (\text{candidate}) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\ h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

Focus on derivative of cell w.r.t previous cell:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t \quad (\text{elementwise}).$$

So the multi-step derivative along the cell state path is:

$$\frac{\partial c_t}{\partial c_{t-k}} = \prod_{s=t-k+1}^t f_s.$$

If forget gates f_s are close to 1 (this is achievable by initializing forget-bias positive, e.g. $b_f = 1$ or 2), the product remains near 1 and doesn't vanish. This gives an *error-flow highway* through time which prevents vanishing gradients. Because the cell update is additive (no repeated multiplications by $\mathbf{W}_h$), LSTM avoids repeated multiplications by matrices that can shrink gradients.

Step 7 — practical initialization tip. Common practice: initialize forget gate bias to a positive value so f_t initially near 1 (e.g., $b_f = 1$), helping gradients to propagate.

Marking guideline (5 marks):

- Correct expression for single-step Jacobian $\mathbf{J}_s$ (2 marks), Show product form and mention norm/ spectral effect (2 marks), Explain LSTM cell derivative chain and effect of f_t (1 mark).

Q.3b Derive the gating equations of a GRU and show how its update rule can be interpreted as a convex combination of the previous hidden state and candidate hidden state. Compare mathematically why GRUs tend to have fewer parameters than LSTMs and discuss scenarios where this might improve generalization in real-world applications.

Solution:

Step 1 — GRU forward equations (component by component). At time t , given input $\mathbf{x}_t$ and previous hidden $\mathbf{h}_{t-1}$:

$$\begin{aligned} z_t &= \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z) \quad (\text{update gate}) \\ r_t &= \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r) \quad (\text{reset gate}) \\ \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (r_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h) \quad (\text{candidate}) \\ \mathbf{h}_t &= (1 - z_t) \odot \mathbf{h}_{t-1} + z_t \odot \tilde{\mathbf{h}}_t. \end{aligned}$$

All operations are elementwise; σ is logistic sigmoid so values in $(0, 1)$.

Step 2 — convex combination proof. For each hidden dimension i :

$$h_{t,i} = (1 - z_{t,i})h_{t-1,i} + z_{t,i}\tilde{h}_{t,i}.$$

Because $0 < z_{t,i} < 1$, weights $(1 - z_{t,i}), z_{t,i}$ are non-negative and sum to 1:

$$(1 - z_{t,i}) + z_{t,i} = 1.$$

Hence $h_{t,i}$ is a convex combination (interpolation) between old state and candidate.

Step 3 — parameter count comparison (explicit). Let input dimension D and hidden size H . We count the major weight matrices.

GRU:

- $\mathbf{W}_z : H \times D, \mathbf{U}_z : H \times H, \mathbf{W}_r : H \times D, \mathbf{U}_r : H \times H, \mathbf{W}_h : H \times D, \mathbf{U}_h : H \times H$

Total parameters (weights) = $3(HD + H^2)$.

LSTM: LSTM has input, forget, output gates and candidate (4 blocks):

- For gates and candidate: each has W_* of size $H \times D$ and U_* of size $H \times H$.

Total parameters = $4(HD + H^2)$.

So GRU uses about 3/4 of LSTM's weights (roughly 25% fewer). (Exact numbers change if one uses peephole connections, biases counted, or different variants.)

Step 4 — why fewer parameters can help generalization (intuition + math). -

For a fixed size dataset, fewer parameters means lower capacity and reduced overfitting risk. From statistical learning, generalization bounds often scale with model complexity (e.g., VC dimension or norm-based capacity measures).

- In practice: if tasks do not require the extra control provided by LSTM's separate cell + three gates, GRU's simpler structure attains similar performance with fewer parameters $\rightarrow$ better generalization on small data and faster training.

- Conversely, for highly complex long-term dependency tasks, the extra flexibility of LSTM can pay off.

Step 5 — additional comment on gradient flow. Both GRU and LSTM create pathways with multiplicative gating that can be near 1, enabling gradient flow across many timesteps. GRU achieves this via z_t interpolation and r_t reset; LSTM via dedicated cell with forget gate.

Marking guideline (5 marks):

- Equations correctness (2 marks).
- Convex combination explanation (1.5 marks).
- Parameter-count comparison and generalization discussion (1.5 marks).